

Exercise 4 : Lexer and Parser

练习 1

使用ocamllex和ocamlyacc实现对四则运算的词法和语法分析，使用dune构建项目。

1.验证dune环境

```
dune --version
```

以下是问题备忘：vscode使用dune可能遇到的一个问题是，.ml文件莫名其妙爆红，可能的办法是在构建的时候使用以下命令：

```
dune build @ocaml-index
```

如果显示没有ocaml-index的话就运行下面命令安装后重启vscode，再运行上面的构建命令：
opam install ocaml-index

2.创建项目目录，创建dune配置文件

```
mkdir my_project && cd my_project  
mkdir lib  
touch dune dune-project lib/dune
```

3.创建词法和语法分析器文件: lexer.mll, parser.mly, ast.ml

```
(* 创建 lib/lexer.mll *)  
{  
open Parser  
}  
  
rule read = parse  
  | [' ' '\t' '\n'] { read lexbuf } (* 跳过空白符 *)  
  | ['0'-'9']+ as num { INT (int_of_string num) }  
  | '+' { PLUS }  
  | '-' { MINUS }  
  | '*' { TIMES }  
  | '/' { DIV }  
  | eof { EOF }  
  | _ { failwith "Unknown character" }
```

```
// 创建 lib/parser.mly  
%{  
  open Ast  
%}  
  
%token <int> INT  
%token PLUS MINUS TIMES DIV EOF  
%start main  
%type <Ast.expr> main  
%%
```

```

main:
  expr EOF { $1 }
;

expr:
  | term PLUS expr { Add($1, $3) }
  | term MINUS expr { Sub($1, $3) }
  | term { $1 }
;

term:
  | factor TIMES term { Mul($1, $3) }
  | factor DIV term { Div($1, $3) }
  | factor { $1 }
;

factor:
  | INT { Num($1) }
;

```

```

(* 创建 lib/ast.ml *)
type expr =
  | Num of int
  | Add of expr * expr
  | Sub of expr * expr
  | Mul of expr * expr
  | Div of expr * expr

```

4.创建main.ml

```

(* 创建 main.ml *)
open Lex_project
open Ast

let rec string_of_expr = function
  | Num n -> string_of_int n
  | Add (e1, e2) -> Printf.sprintf "(%s + %s)" (string_of_expr e1)
    (string_of_expr e2)
  | Sub (e1, e2) -> Printf.sprintf "(%s - %s)" (string_of_expr e1)
    (string_of_expr e2)
  | Mul (e1, e2) -> Printf.sprintf "(%s * %s)" (string_of_expr e1)
    (string_of_expr e2)
  | Div (e1, e2) -> Printf.sprintf "(%s / %s)" (string_of_expr e1)
    (string_of_expr e2)

let () =
  let lexbuf = Lexing.from_channel stdin in
  let ast = Parser.main Lexer.read lexbuf in
  Printf.printf "\nAST: %s\n" (string_of_expr ast)

```

5.写dune配置

```
; lib/dune
(library
  (name lex_project)
  (modules lexer ast parser))

; dune
(executable
  (name main)
  (libraries lex_project))

; dune-project
(lang dune 3.0)
```

6.使用ocamllex和ocamlyacc生成词法和语法文件，dune编译并运行

```
ocamllex lib/lexer.mll
ocamlyacc -b lib/parser lib/parser.mly

dune build
./_build/default/main.exe
```

运行后输入四则运算如 $1 + 2 * 3 - 4$ ，按下ctrl + D作为EOF结束符，看看生成的AST是否正确保证了运算优先级。

练习2

现有一个文法 $G[E]$ 如下，请你为其实现递归下降语法分析，实现parse函数，检验一个char list是否合法。

```
G[E]:
E->aAB | bB
A->aB | b
B->bA | a
```

注意：

- 你不需要设计AST数据结构进行存储，只需要判断一个字符串是否可以由该文法产生，输出“Success”或“Failed”。
- 思考 $E \rightarrow aAB$ 的处理方式

示例输入：

```
let test_input = ['a'; 'a'; 'a'; 'a']
```

示例输出：

```
Success
```

练习3

下面是一个简单的函数式语言的语法，请你任选两种语法分析方法之一：

- ocaml yacc
- 递归下降语法分析

实现一个词法和语法分析器，能够为该语法的任一合法句子生成所对应的抽象语法树AST。

t ::=	terms
true	constant true
false	constant false
if t then t else t	conditional
0	constant zero
succ t	successor
pred t	predecessor
iszero t	zero test

参考代码: <https://www.cis.upenn.edu/~bcpierce/tapl/checkers/arith/>

参考书籍: 《Types and Programming Languages》

示例输入:

```
if false then 0 else succ 0

iszero pred succ 0
```

注意:

- 你可以自定义输入输出方式, 如参考练习1一样使用命令行输入
- 你可以实现自己的AST打印函数, 以检查生成效果
- 建议预留扩展词法语法分析的接口